

```

import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getAuth, onAuthStateChanged } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
import { getDatabase, ref, set, onValue, remove, get, child, update } from
"https://www.gstatic.com/firebasejs/10.12.0/firebase-database.js";

const firebaseConfig = {
  apiKey: "AIzaSyBOgShBOu05UzszCBLS-bpTl2f3AI7_I-pY",
  authDomain: "reservasisd.firebaseio.com",
  databaseURL: "https://reservasisd-default-rtdb.firebaseio.com",
  projectId: "reservasisd",
  storageBucket: "reservasisd.firebaseio.com",
  messagingSenderId: "637702189208",
  appId: "1:637702189208:web:49ff477b35e299564ca0ed"
};

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getDatabase(app);
const configHoraReporte = document.getElementById('config-hora-reporte');
const btnGuardarHora = document.getElementById('btn-guardar-hora');
const txtStatusConfig = document.getElementById('txt-status-config');
const selectIntentos = document.getElementById('select-intentos-acceso');

// =====
// CLAVE DE API DE IMGBB
// =====
const IMGBB_API_KEY = "85fcef48792a118d9543ddc94c116ab";

// ELEMENTOS DEL DOM
const formNuevo = document.getElementById('form-nuevo-recurso');
const listaInventario = document.getElementById('lista-inventario');
const btnVolver = document.getElementById('btn-volver');

// NUEVOS ELEMENTOS DEL DOM PARA ADMINS
const formNuevoAdmin = document.getElementById('form-nuevo-admin');
const listaAdministradores = document.getElementById('lista-administradores');

// NUEVOS ELEMENTOS DEL DOM PARA PROFESORES
const formNuevoProfesor = document.getElementById('form-nuevo-profesor');
const listaProfesores = document.getElementById('lista-profesores');

// 1. GUARDIAN Estricto Dinámico
onAuthStateChanged(auth, async (user) => {
  if (!user) {
    window.location.href = "index.html";
    return;
  }
  const email = user.email;
  // EXCEPCIÓN DIRECTA:
  if (email === "laggerro2@gmail.com") {
    inicializarPanel();
  }

```

```

        return;
    }
    // Para cualquier otro correo, verificamos en la RTDB
    try {
        const emailLimpio = email.replace(/\.\/g, '_');
        const dbRef = ref(db);
        const snapshotAdmin = await get(child(dbRef,
`administradores/${emailLimpio}`));
        if (snapshotAdmin.exists()) {
            inicializarPanel();
        } else {
            alert("Acceso denegado. Se requieren credenciales de
administrador.");
            window.location.href = "equipos.html";
        }
    } catch (error) {
        console.error("Error al verificar permisos de administrador:", error);
        window.location.href = "equipos.html";
    }
});

// Modificamos la función inicializadora para cargar todo
function inicializarPanel() {
    escucharYListarEquipos();
    escucharYListarAdmins();
    escucharYListarProfesores();
    escucharYListarIntentosAcceso(); // <-- AGREGAR ESTA LÍNEA
    cargarConfiguracionHora();
}

// =====
// MÓDULO A: CONTROL DE INVENTARIO (EQUIPOS)
// =====

// ENVIAR FORMULARIO (SUBIDA A IMGBB + REGISTRO FIREBASE)
formNuevo.addEventListener('submit', async (e) => {
    e.preventDefault();
    const categoria = document.getElementById('recurso-categoria').value;
    const nombre = document.getElementById('recurso-nombre').value.trim();
    const cantidad =
parseInt(document.getElementById('recurso-cantidad').value);
    const descripcion =
document.getElementById('recurso-descripcion').value.trim();
    const inputImagen = document.getElementById('recurso-imagen').files[0];

    const idRecurso = nombre.toLowerCase()
        .replace(/^[a-z0-9]+/g, '_')
        .replace(/(^_|_$)/g, '');

    let urlFinalImagen = "https://i.ibb.co/5z0RJbk/default-recurso.png"; //
Imagen por defecto por seguridad

    try {
        // 1. Subida a ImgBB en lugar del PHP local

```

```

    if (inputImagen) {
        console.log("Subiendo imagen a ImgBB");
        const formData = new FormData();
        formData.append('image', inputImagen);

        const responseImgBB = await
fetch(`https://api.imgbb.com/1/upload?key=${IMGBB_API_KEY}`, {
            method: 'POST',
            body: formData
        });

        const dataImgBB = await responseImgBB.json();

        if (!dataImgBB.success) {
            throw new Error("No se pudo subir la imagen al servidor de
ImgBB.");
        }

        // Guardamos la URL pública directa que nos da ImgBB
        urlFinalImagen = dataImgBB.data.url;
        console.log("Imagen subida con éxito: ", urlFinalImagen);
    }

    // 2. Registro directo en Firebase RTDB
    const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);
    await set(recursoRef, {
        nombre: nombre,
        stock_total: cantidad,
        descripcion: descripcion,
        imagen: urlFinalImagen
    });

    alert("¡Equipo agregado con éxito al inventario!");
    formNuevo.reset();
} catch (error) {
    console.error("Error al registrar:", error);
    alert(`Falló el registro: ${error.message}`);
}
});

// LISTAR EQUIPOS EN AJUSTES.JS
function escucharYListarEquipos() {
    const inventarioRef = ref(db, 'inventario');
    onValue(inventarioRef, (snapshot) => {
        listaInventario.innerHTML = '';
        const categorias = snapshot.val();
        if (!categorias) {
            listaInventario.innerHTML = '<p class="text-center text-gray-400
py-4">No hay equipos en el inventario.</p>';
            return;
        }
        Object.keys(categorias).forEach(idCategoria => {
            const recursos = categorias[idCategoria];

```

```

    Object.keys(recursos).forEach(idIdRecurso => {
      const item = recursos[idIdRecurso];
      const itemHTML = `
        <div class="flex items-center justify-between p-3 bg-gray-50
border rounded-xl hover:shadow-sm transition-all mb-2">
          <div class="flex items-center gap-3">
            
            <div>
              <h4 class="font-bold text-gray-800 text-sm
leading-tight">${item.nombre}</h4>
              <p class="text-xs text-gray-500 uppercase
tracking-wide font-semibold">${idCategoria}</p>
            </div>
          </div>

          <!-- Control de Stock Interactivo -->
          <div class="flex items-center gap-2">
            <span class="text-xs text-gray-500
font-bold">Stock:</span>
            <div class="flex items-center bg-white border rounded-lg
overflow-hidden shadow-sm">
              <button onclick="cambiarStock('${idCategoria}',
'${idIdRecurso}', ${item.stock_total}, -1)" class="px-2 py-1 bg-gray-100
hover:bg-gray-200 text-gray-600 font-bold transition text-xs">-</button>
              <span class="px-3 font-bold text-gray-800
text-xs">${item.stock_total}</span>
              <button onclick="cambiarStock('${idCategoria}',
'${idIdRecurso}', ${item.stock_total}, 1)" class="px-2 py-1 bg-gray-100
hover:bg-gray-200 text-gray-600 font-bold transition text-xs">+</button>
            </div>

            <button onclick="eliminarRecurso('${idCategoria}',
'${idIdRecurso}')" class="text-red-500 hover:text-red-700 p-2 transition ml-2">
              <i class="fa-solid fa-trash-can"></i>
            </button>
          </div>
        </div>
      `;
      listaInventario.insertAdjacentHTML('beforeend', itemHTML);
    });
  });
});
}

// NUEVA FUNCIÓN GLOBAL PARA ACTUALIZAR EL STOCK EN LA DB
window.cambiarStock = async function (categoria, idRecurso, stockActual, cambio)
{
  const nuevoStock = stockActual + cambio;
  if (nuevoStock < 0) return; // Evitamos stock negativo

  try {
    const updates = {};
    updates[`inventario/${categoria}/${idRecurso}/stock_total`] =

```

```

nuevoStock;
    await update(ref(db), updates);
    console.log(`Stock actualizado para ${idRecurso}: ${nuevoStock}`);
  } catch (error) {
    console.error("Error al actualizar stock:", error);
    alert("No se pudo actualizar el stock del equipo.");
  }
};

// ELIMINAR RECURSO
window.eliminarRecurso = async function (categoria, idRecurso) {
  const confirmar = confirm("¿Estás seguro de que quieres eliminar este recurso del inventario de forma permanente? Se perderán sus datos de stock.");
  if (confirmar) {
    try {
      const recursoRef = ref(db, `inventario/${categoria}/${idRecurso}`);
      await remove(recursoRef);
      alert("Recurso eliminado del inventario con éxito.");
    } catch (error) {
      console.error("Error al eliminar:", error);
      alert("No se pudo eliminar el recurso.");
    }
  }
};

// =====
// MÓDULO B: CONTROL DE ADMINISTRADORES
// =====

// AGREGAR NUEVO ADMINISTRADOR
formNuevoAdmin.addEventListener('submit', async (e) => {
  e.preventDefault();
  const nombre = document.getElementById('admin-nombre').value.trim();
  const email =
document.getElementById('admin-email').value.trim().toLowerCase();
  const emailLimpio = email.replace(/\.\/g, '_');
  try {
    const adminRef = ref(db, `administradores/${emailLimpio}`);
    await set(adminRef, {
      nombre: nombre,
      email: email
    });
    alert(`Se han asignado privilegios de administrador a ${nombre} de manera exitosa.`);
    formNuevoAdmin.reset();
  } catch (error) {
    console.error("Error al agregar administrador:", error);
    alert("Ocurrió un error al registrar el administrador en la base de datos.");
  }
});

// ESCUCHAR Y LISTAR ADMINISTRADORES
function escucharYListarAdmins() {

```

```

const adminsRef = ref(db, 'administradores');
onValue(adminsRef, (snapshot) => {
  listaAdministradores.innerHTML = '';
  const admins = snapshot.val();
  if (!admins) {
    listaAdministradores.innerHTML = '<p class="text-center text-gray-400 py-4">No hay administradores registrados.</p>';
    return;
  }
  Object.keys(admins).forEach(key => {
    const admin = admins[key];
    const esUsuarioPrincipal = admin.email === "laggerro2@gmail.com";
    const botonBorrar = esUsuarioPrincipal
      ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5 rounded-full font-semibold uppercase">Admin Principal</span>`
      : `<button onclick="eliminarAdmin('${key}', '${admin.nombre}')" class="text-red-500 hover:text-red-700 p-2 transition">
        <i class="fa-solid fa-user-minus"></i>
      </button>`;
    const itemHTML = `
    <div class="flex items-center justify-between p-3 bg-gray-50 border rounded-xl hover:shadow-sm transition-all mb-2">
      <div class="flex items-center gap-3">
        <div class="w-10 h-10 bg-indigo-100 rounded-full flex items-center justify-center text-indigo-700">
          <i class="fa-solid fa-user-shield"></i>
        </div>
        <div>
          <h4 class="font-bold text-gray-800 text-sm leading-tight">${admin.nombre}</h4>
          <p class="text-xs text-gray-500 font-medium">${admin.email}</p>
        </div>
      </div>
      ${botonBorrar}
    </div>
  `;
    listaAdministradores.insertAdjacentHTML('beforeend', itemHTML);
  });
});
}

// BORRAR ADMINISTRADOR
window.eliminarAdmin = async function (emailLimpio, nombreAdmin) {
  const confirmar = confirm(`¿Estás seguro de que quieres retirarle los permisos de administrador a "${nombreAdmin}"? \n Esta cuenta ya no podrá acceder a este panel de ajustes ni ver el botón de configuración.`);
  if (confirmar) {
    try {
      const adminRef = ref(db, `administradores/${emailLimpio}`);
      await remove(adminRef);
      alert(`Permisos revocados para ${nombreAdmin} correctamente.`);
    } catch (error) {
      console.error("Error al revocar permisos:", error);
    }
  }
}

```

```

        alert("No se pudo dar de baja el administrador.");
    }
}
};

// BOTÓN VOLVER
btnVolver.addEventListener('click', () => {
    window.location.href = "equipos.html";
});

// =====
// MÓDULO C: CONTROL DE PROFESORES
// =====

// REGISTRAR PROFESOR EN LA LISTA BLANCA
formNuevoProfesor.addEventListener('submit', async (e) => {
    e.preventDefault();
    const nombre = document.getElementById('profe-nombre').value.trim();
    const email =
document.getElementById('profe-email').value.trim().toLowerCase();
    const emailLimpio = email.replace(/\.\/g, '_');

    try {
        // 1. Guardar en usuarios_autorizados
        const profeRef = ref(db, `usuarios_autorizados/${emailLimpio}`);
        await set(profeRef, {
            nombre: nombre,
            email: email,
            recibe_reporte: false
        });

        // 2. Si estaba en la lista de intentos pendientes, lo eliminamos
        const intentoRef = ref(db, `intentos_acceso/${emailLimpio}`);
        await remove(intentoRef);

        alert(`El docente ${nombre} ha sido agregado a la lista autorizada.`);
        formNuevoProfesor.reset();
    } catch (error) {
        console.error("Error al registrar profesor:", error);
        alert("Ocurrió un error al guardar el docente.");
    }
});

// ESCUCHAR Y POPULAR EL SELECT DE INTENTOS DE ACCESO
function escucharYListarIntentosAcceso() {
    if (!selectIntentos) return;
    const intentosRef = ref(db, 'intentos_acceso');

    onValue(intentosRef, (snapshot) => {
        selectIntentos.innerHTML = '<option value="">-- Seleccionar usuario no
autorizado --</option>';
        const intentos = snapshot.val();

```

```

    if (intentos) {
        Object.keys(intentos).forEach(key => {
            const item = intentos[key];
            const option = document.createElement('option');
            option.value = key;
            // Guardamos los datos en atributos dataset para completarlos
fácil
            option.dataset.email = item.email || '';
            option.dataset.nombre = item.nombre || item.email;
            option.textContent = `${item.nombre ? item.nombre + ' (' +
item.email + ')' : item.email}`;
            selectIntentos.appendChild(option);
        });
    }
});

// Evento al cambiar la opción seleccionada
selectIntentos.addEventListener('change', (e) => {
    const selectedOption = e.target.options[e.target.selectedIndex];
    if (selectedOption && selectedOption.value !== "") {
        document.getElementById('profe-nombre').value =
selectedOption.dataset.nombre || '';
        document.getElementById('profe-email').value =
selectedOption.dataset.email || '';
    } else {
        document.getElementById('profe-nombre').value = '';
        document.getElementById('profe-email').value = '';
    }
});
}

// ESCUCHAR Y LISTAR PROFESORES
function escucharYListarProfesores() {
    const profesoresRef = ref(db, 'usuarios_autorizados');
    onValue(profesoresRef, (snapshot) => {
        listaProfesores.innerHTML = '';
        const profesores = snapshot.val();
        if (!profesores) {
            listaProfesores.innerHTML = '<p class="text-center text-gray-400
py-4">No hay docentes autorizados en la lista.</p>';
            return;
        }
        Object.keys(profesores).forEach(key => {
            const profe = profesores[key];
            const esPropietario = profe.email === "laggerro2@gmail.com";
            const botonBorrar = esPropietario
                ? `<span class="text-xs bg-blue-100 text-blue-800 px-2.5 py-0.5
rounded-full font-semibold">Admin Principal</span>`
                : `<button onclick="eliminarProfesor('${key}',
'${profe.nombre}')" class="text-red-500 hover:text-red-700 p-2 transition">
                    <i class="fa-solid fa-user-slash"></i>
                </button>`;

```



```

const itemHTML = `
  <div class="flex items-center justify-between p-3 bg-gray-50 border
rounded-xl hover:shadow-sm transition-all mb-2">
    <div class="flex items-center gap-3">
      <div class="w-10 h-10 bg-green-100 rounded-full flex
items-center justify-center text-green-700">
        <i class="fa-solid fa-chalkboard-user"></i>
      </div>
      <div>
        <h4 class="font-bold text-gray-800 text-sm
leading-tight">${profe.nombre}</h4>
        <p class="text-xs text-gray-500
font-medium">${profe.email}</p>
        <label class="inline-flex items-center gap-1.5 mt-1
cursor-pointer select-none">
          <input type="checkbox" id="check-rep-${key}"
class="w-3.5 h-3.5 rounded text-green-600 focus:ring-green-500 cursor-pointer"
${profe.recibe_reporte ? 'checked' : ''}>
          <span class="text-[10px] text-gray-500 font-bold
uppercase tracking-wider">¿Recibe Reporte Diario?</span>
        </label>
      </div>
    </div>
    <div>
      <button class="text-gray-500 font-medium">Borrar</button>
    </div>
  </div>
`;

listaProfesores.insertAdjacentHTML('beforeend', itemHTML);

const checkReporte = document.getElementById(`check-rep-${key}`);
if (checkReporte) {
  checkReporte.addEventListener('change', async (e) => {
    const nuevoEstado = e.target.checked;
    try {
      const updates = {};
      updates[`usuarios_autorizados/${key}/recibe_reporte`] =
nuevoEstado;

      await update(ref(db), updates);
      console.log(`Preferencia de reporte actualizada para
${profe.nombre}: ${nuevoEstado}`);
    } catch (err) {
      console.error("Error al actualizar preferencia de
reporte:", err);

      alert("No se pudo actualizar la configuración.");
      e.target.checked = !nuevoEstado; // Revierte el check
    }
  });
}

});
});
});
}

// QUITAR PROFESOR DE LA LISTA BLANCA
window.eliminarProfesor = async function (emailLimpio, nombreProfe) {
  const confirmar = confirm(`¿Estás seguro de que querés REVOCAR el acceso a

```

```

"${nombreProfe}"?\nSi lo hacés, perderá la capacidad de iniciar sesión de forma
inmediata.`);
    if (confirmar) {
        try {
            const profeRef = ref(db, `usuarios_autorizados/${emailLimpio}`);
            await remove(profeRef);
            const adminRef = ref(db, `administradores/${emailLimpio}`);
            await remove(adminRef);
            alert(`Acceso revocado con éxito para ${nombreProfe}.`);
        } catch (error) {
            console.error("Error al revocar acceso:", error);
            alert("No se pudo completar la operación.");
        }
    }
};

// =====
// MÓDULO D: CONFIGURACIÓN GENERAL DEL REPORTE
// =====

// 1. Leer la hora guardada en Firebase y rellenar el input
async function cargarConfiguracionHora() {
    try {

        const configRef = ref(db, 'configuracion/hora_reporte');
        const snapshot = await get(configRef);

        if (snapshot.exists()) {
            configHoraReporte.value = snapshot.val();
            console.log("Hora de reporte cargada desde la BD:", snapshot.val());
        } else {
            configHoraReporte.value = "08:00"; // Hora por defecto
            console.log("No se encontró hora configurada. Se estableció 08:00
por defecto.");
        }
    } catch (error) {
        console.error("Error al cargar la hora del reporte:", error);
    }
}

// 2. Guardar la nueva hora seleccionada por el Administrador
btnGuardarHora.addEventListener('click', async (e) => {
    e.preventDefault(); // Evita recargas
    const nuevaHora = configHoraReporte.value.trim();
    if (!nuevaHora) {
        alert("Por favor, ingresá al menos un horario válido.");
        return;
    }

    try {
        btnGuardarHora.disabled = true;
        btnGuardarHora.innerHTML = `

```

```

    // directamente al nodo de configuración
    const horaRef = ref(db, 'configuracion/hora_reporte');
    await set(horaRef, nuevaHora);

    // PROTECCIÓN: Solo manejamos la clase si el elemento realmente existe
    en el HTML
    if (txtStatusConfig) {
        txtStatusConfig.classList.remove('hidden');
        setTimeout(() => {
            txtStatusConfig.classList.add('hidden');
        }, 3000);
    } else {
        // Si no existe el elemento mostrar un alert simple
        alert("¡Horarios guardados con éxito!");
    }

    } catch (error) {
        console.error("Error al guardar la configuración de hora:", error);
        alert("No se pudo guardar la configuración en la base de datos.");
    } finally {
        btnGuardarHora.disabled = false;
        btnGuardarHora.innerHTML = `<i class="fa-solid fa-floppy-disk"></i>
Guardar Horarios`;
    }
});

```